

DICE ROLLING SIMULATOR

Python Project Report

Project Type: Beginner Python Programming Project

Programming Language: Python 3

Main File: Dice.py

This report documents a simple command-line dice rolling simulator. The program generates a random value from 1 to 6, displays the corresponding ASCII-art die face, and allows the user to roll again.

1. Introduction

A dice rolling simulator is a small programming exercise that demonstrates how random numbers can be used to model a familiar real-world process. A standard six-sided die has six possible outcomes: 1, 2, 3, 4, 5 and 6. For a fair die, each outcome has an equal theoretical probability of $1/6$, or approximately 16.67%.

This project implements that idea in Python using the random-number functionality available in Python's standard library.

2. Objectives

The main objectives of the project are to:

- Generate a random die value from 1 to 6.
- Display the result using ASCII art.
- Allow repeated rolls based on user input.
- Practice functions, conditions, input and random number generation.
- Examine whether many simulated rolls approach the expected distribution of a fair die.

3. Tools and Technologies

Python 3: Used to implement the simulator.

random module: Used to generate the die outcome.

Command-line interface: Used for interaction with the user.

ASCII art: Used to represent the six die faces.

4. Program Methodology

The program follows a simple sequence:

1. Call the **roll()** function.
2. Generate an integer between 1 and 6.
3. Use conditional statements to identify the generated face.
4. Print the matching ASCII-art representation.
5. Ask the user whether another roll is required.
6. If the response is **y**, call the function again; otherwise, finish.

5. Important Code

```
from random import *

def roll():
    no = randint(1, 6)

    if(no == 1):
        print("          [-----]\n"
              "          |         |\n"
              "          |    o    |\n"
              "          |         |\n"
              "          [-----]")

    # Similar conditional blocks display faces 2 through 6.

roll()

ans = input(" Would you like to roll again\nType 'y' or 'n' ")

if(ans == 'y'):
    roll()
else:
    print('\n')
```

6. Code Explanation

The uploaded program imports the random functionality and defines a function named **roll()**. Inside the function, **randint(1, 6)** generates the simulated die result. The program then uses six conditional blocks to print a visual representation of the corresponding face. After the first roll, user input determines whether another roll is made.

The current source uses recursion for repetition: when the user enters **y**, **roll()** is called again. This is suitable for demonstrating functions, although a loop would generally be preferable for a production-quality simulator.

7. What Makes the Simulation Fair?

The simulator chooses an integer from 1 through 6 using Python's pseudo-random number generator. Under the intended uniform selection, the six outcomes have the same theoretical probability. Therefore:

Expected probability of each face = $1/6 \approx 16.67\%$

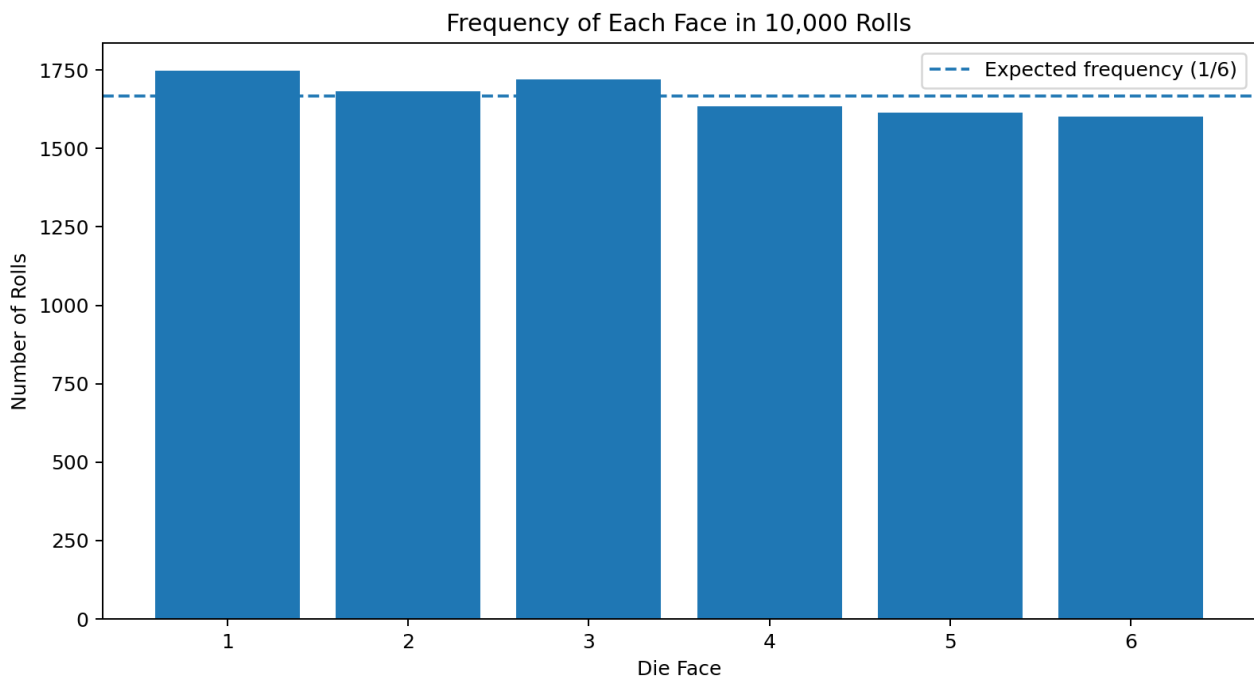
Fairness should not be judged from only a few rolls. For example, obtaining several 6s in a row is possible even when the underlying process is fair. A larger sample provides a better way to observe the overall distribution.

8. Statistical Experiment — 10,000 Rolls

To demonstrate the expected behavior, a separate statistical experiment was performed with 10,000 simulated rolls. The following results are illustrative of a large random sample and are included to show how frequencies can be compared with the theoretical expectation.

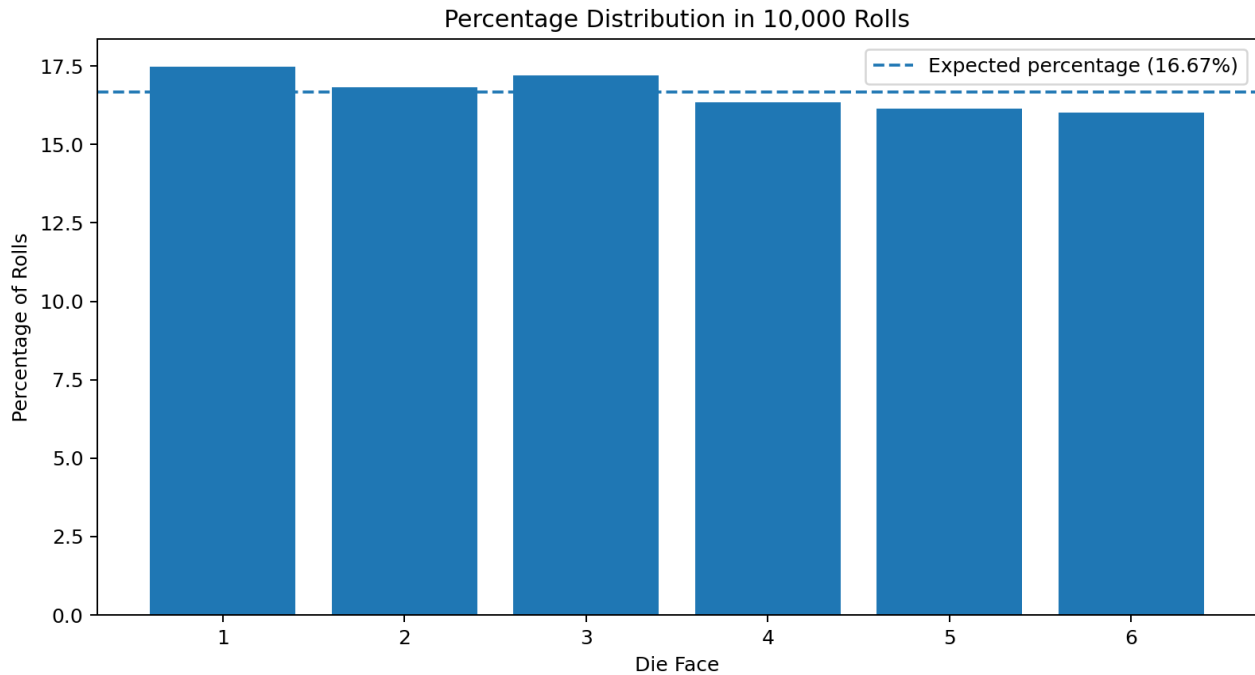
Face	Observed Rolls	Observed %	Expected %
1	1,748	17.48%	16.67%
2	1,683	16.83%	16.67%
3	1,720	17.20%	16.67%
4	1,634	16.34%	16.67%
5	1,613	16.13%	16.67%
6	1,602	16.02%	16.67%

9. Frequency Distribution



The dashed reference line represents the theoretical expected frequency of 1,666.67 occurrences per face. Random variation means the observed bars will not normally be identical, but a large sample should place them in the same general range.

10. Percentage Distribution



The percentage chart compares the observed share of each face with the theoretical 16.67% share. Small deviations are expected in a random experiment. As the number of rolls increases, the observed proportions generally tend to move closer to the theoretical probabilities.

11. Interpretation of Results

The statistical experiment illustrates an important distinction between **randomness** and **equal long-run probabilities**. A fair random process does not need to produce exactly the same number of 1s, 2s, 3s, 4s, 5s and 6s in every sample. Instead, random fluctuations are expected. With more observations, the relative frequencies tend to stabilize around their theoretical values.

Therefore, the 10,000-roll experiment provides a practical demonstration that is consistent with the expected behavior of a uniform six-sided random simulation. It is evidence of the distribution observed in the experiment, rather than a mathematical proof that every individual roll is physically fair.

12. Limitations

The current implementation is intentionally simple. It does not record statistics during normal interactive use, has limited input validation, and uses recursive calls to repeat the rolling process. The project also relies on pseudo-randomness rather than a physical die.

13. Future Improvements

Possible improvements include:

- Replace recursion with a loop.
- Add a roll counter and frequency table.
- Let the user choose how many rolls to perform.
- Automatically calculate probabilities and percentages.
- Generate charts from the program itself.
- Validate invalid user input.
- Add automated unit tests.
- Support dice with different numbers of sides.

14. Conclusion

The Dice Rolling Simulator is a compact Python project that demonstrates fundamental programming concepts through a familiar probability problem. It generates one of six possible outcomes, displays the result clearly, and supports repeated interaction. The large-sample experiment further demonstrates how a uniform random process can produce frequencies that approximate the theoretical 16.67% share for each face while still showing normal random variation.

End of Project Report